

LibTorch: Day 4

Understanding matrix operations, but before that, why do we need matrices at all for AI?

In computer science, matrix is just a 2D data structure (an array of arrays). But in Deep Learning, matrices are physical engine of Intelligence.

★ What a Neural Network really is?

↓
1) When we pass an image of a warehouse floor into ROS2 robot's camera, the AI doesn't see a picture. It sees a massive matrix of pixel numbers.

2) The "brain" of AI, everything it has learned is completely stored as massive grids of numbers called weights.

↳ When image data passes through a layer of neural network, the computer does a massive mathematical operation:

→ Matrix Multiplication

(Outputs = Input × Weight)

By structuring data as matrices, LibTorch can send the entire grid of memory straight to hardware (CPU/GPU) to multiply them all simultaneously (called vectorization).

// — Matrix Operations in LibTorch —

```
auto mat1 = torch::randn({3, 4});  
auto mat2 = torch::randn({4, 5});
```

-- For matrix multiplication, we use

```
auto matmul = torch::mm(mat1, mat2);
```

OR

```
auto t_matmul = mat1.matmul(mat2);
```

```
auto transposed = mat1.t();    // Transpose:  
                                {4, 3}
```

Reduction Operations

```
auto t = torch::rand({4, 4});
```

```
auto total = t.sum();
```

// Sum of all elements

```
auto row_sum = t.sum(1);
```

// Sum along dimension 1

```
auto col_mean = t.mean(0);
```

// Mean along dimension 0

```
auto max_val = t.max();
```

```
auto min_val = t.min();
```

Adding OR Removing Dimensions

```
auto v = torch::ones({3});
```

```
auto row_vec = v.unsqueeze(0); // Shape: {1, 3}
```

```
auto col_vec = v.unsqueeze(1); // Shape: {3, 1}
```

```
auto back = col_vec.squeeze(1); // Back to {3}
```